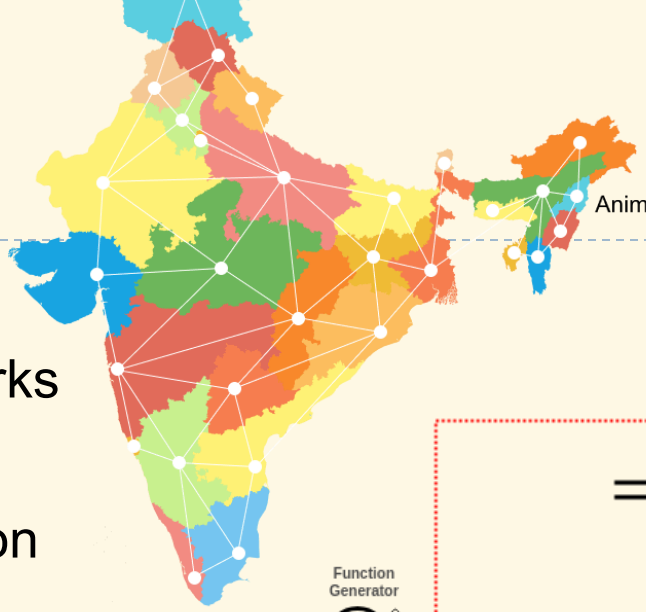
Graphs

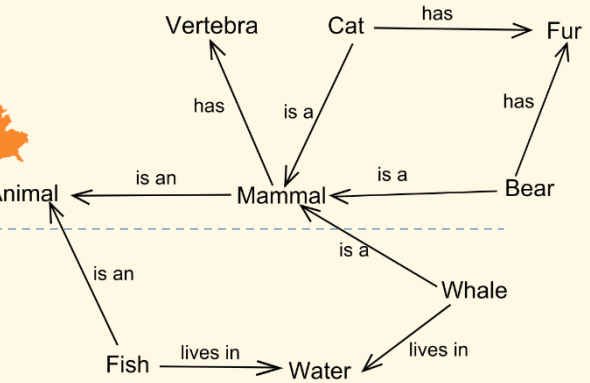
- Definitions
- Implementation
- Traversal, search, shortest path

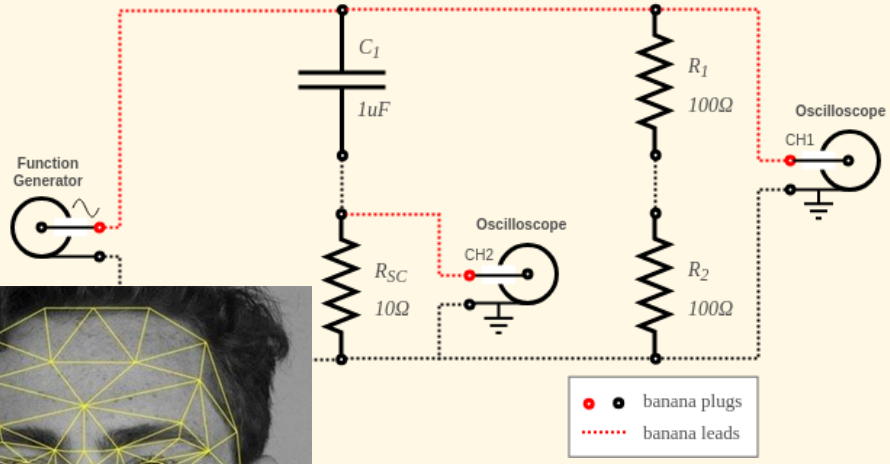
Overview of Graphs

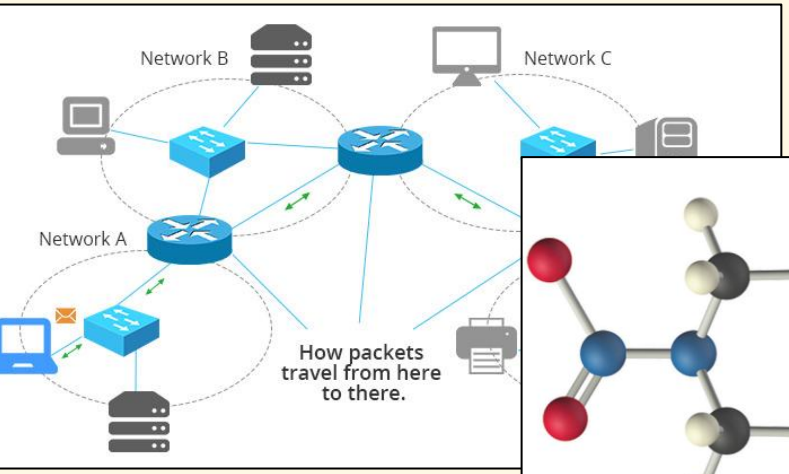
Introduction

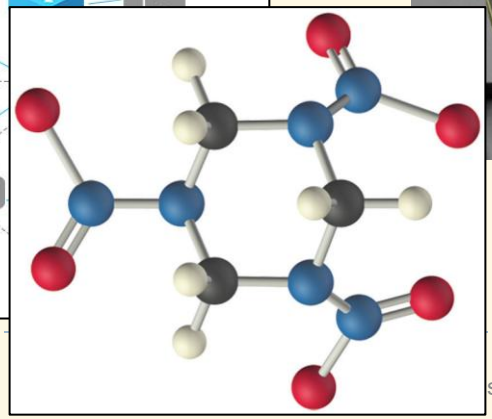
- Traffic networks
- Communication networks
- Social networks
- Semantic representation
- Electrical circuits
- Chemical molecular structures
- …

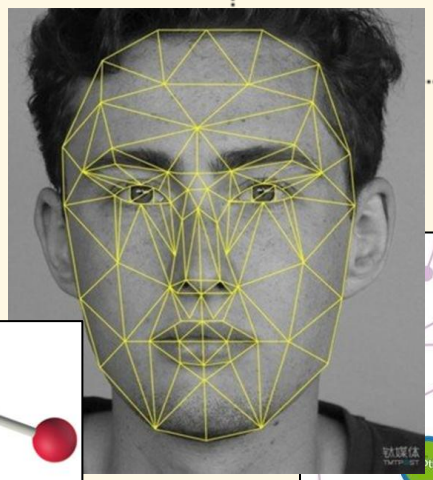


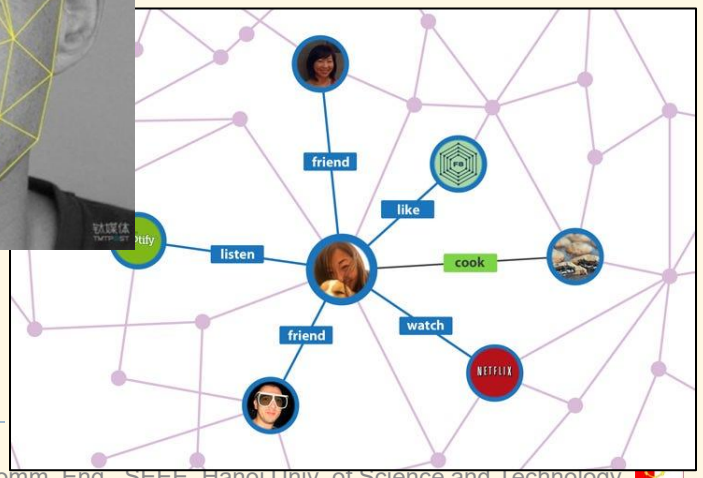






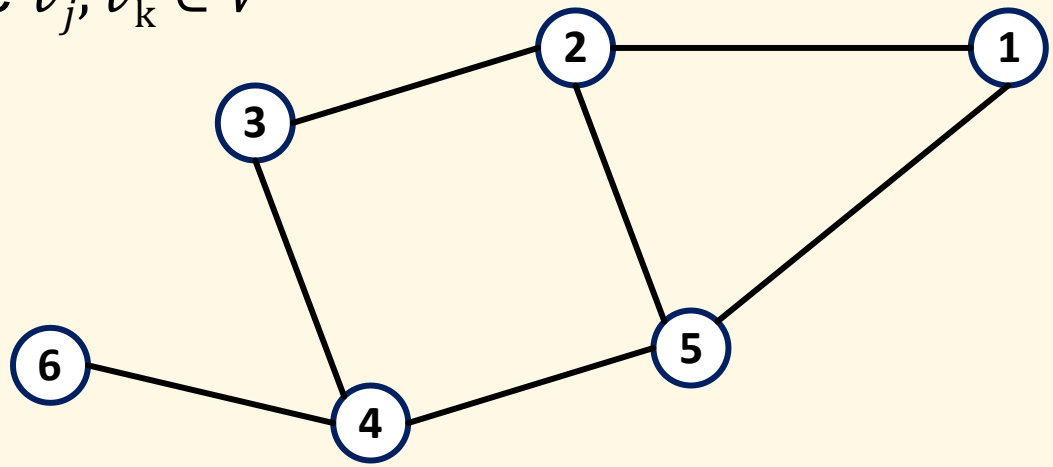






Definitions

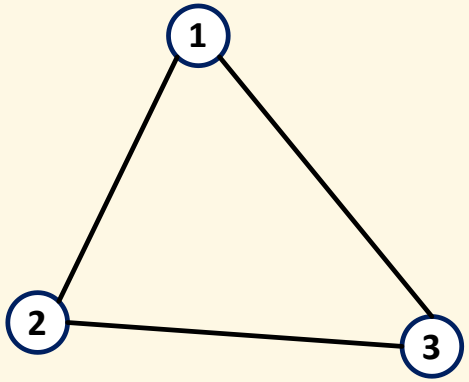
- Graph is a pair: $G = \langle V, E \rangle$, where:
  - $V = \{v_1, v_2, \ldots, v_n\}$ are *vertices*, or nodes
  - $E = \{e_1, e_2, \ldots, e_m\}$ are *edges*, or connections
  - $e_i = \langle v_j, v_k \rangle$ where $v_j, v_k \in V$



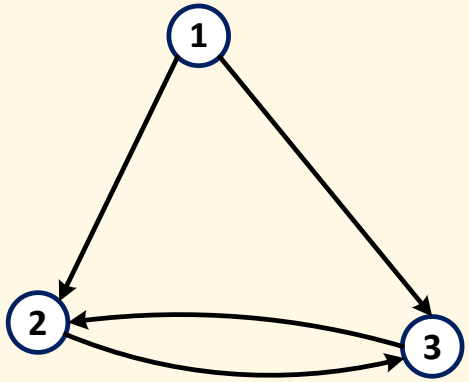
- Example
  - $V = \{1,2,3,4,5,6\}$
  - $E = \{\langle 1,2 \rangle, \langle 2,3 \rangle, \langle 2,5 \rangle, \langle 3,4 \rangle, \langle 4,5 \rangle, \langle 4,6 \rangle\}$
- Trees are graphs

Directivity

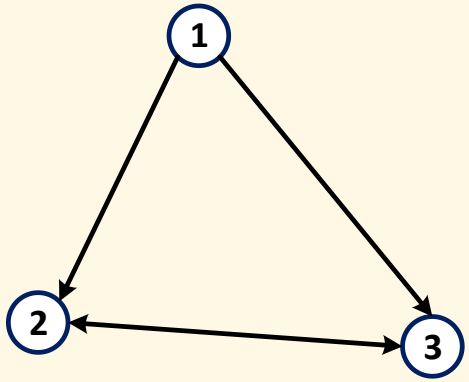
- *Undirected graph*: edges are unordered
  - $E_1 = \{\langle 1,2\rangle, \langle 1,3\rangle, \langle 2,3\rangle\}$
  - $\langle u, v\rangle \in E$ implies that $\langle v, u\rangle \in E$
- *Directed graph*: edges are ordered
  - $E_2 = \{\langle 1,2\rangle, \langle 1,3\rangle, \langle 2,3\rangle, \langle 3,2\rangle\}$
- Trees can be either directed or undirected



G_1: undirected



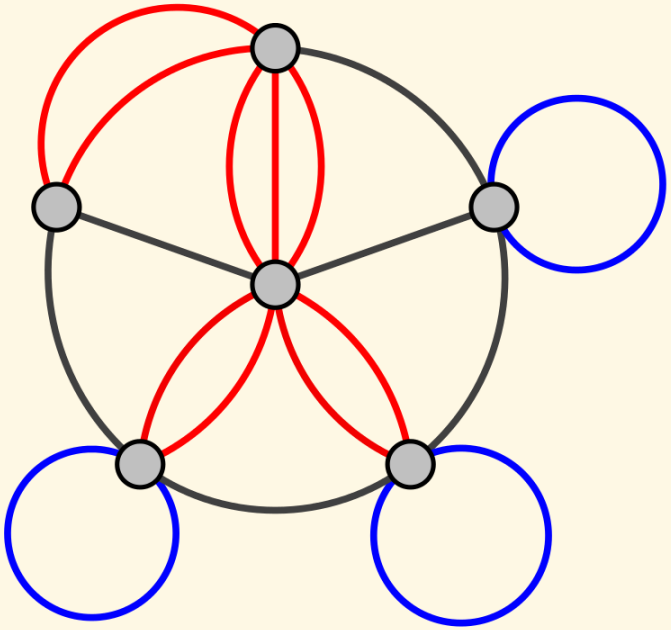
G_2: directed



G_2: directed with simplified drawing

Self-edges, multi-edges

- A *self-edge* (aka loop edge) is an edge where its two vertices are identical: $\langle u, u \rangle$
- A *multi-edge* (aka parallel edge) is an edge that appears multiple times
- A graph is called *simple* iff it has no self-edge neither multi-edge
  - Otherwise, the graph is called *multi-graph*

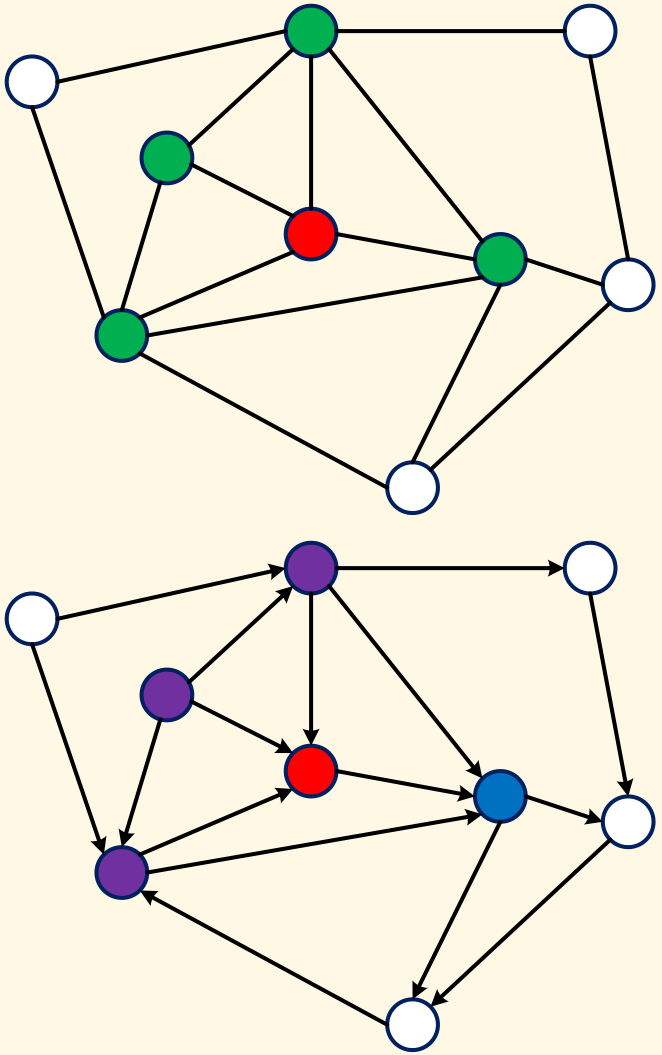


A multi-graph with multi-edges and self-edges

- In normal convention, if no further note given, only simple graphs are considered

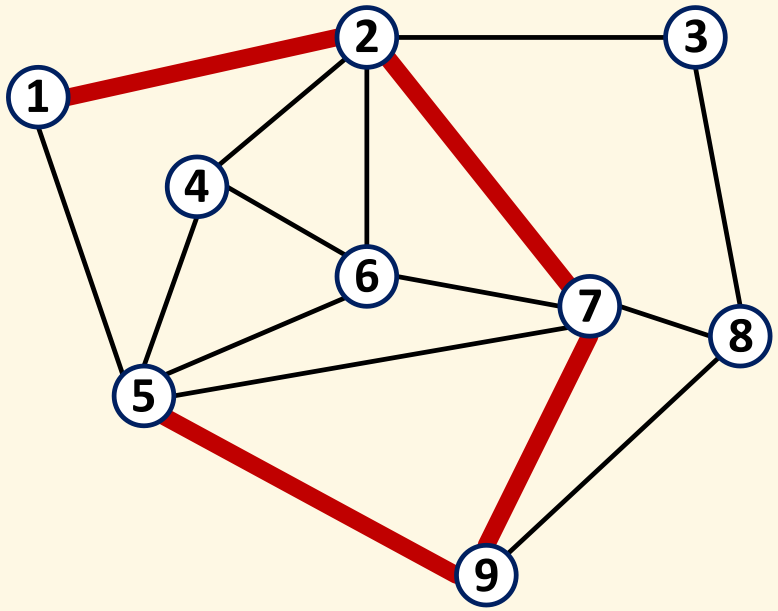
Neighborhood

- In undirected graph: vertex u is *neighbor* of v iff there exists an edge composed of those two vertices
  - *Degree* of a vertex: number of its neighbors
- In directed graph:
  - u is an *out-neighbor* of v iff there exists an edge $\langle v, u \rangle$
  - u is an *in-neighbor* of v iff there exists an edge $\langle u, v \rangle$
  - u is *neighbor* of v iff u is either an out-neighbor or an in-neighbor of v
  - *In-degree*, *out-degree*: numbers of in- and out-neighbors



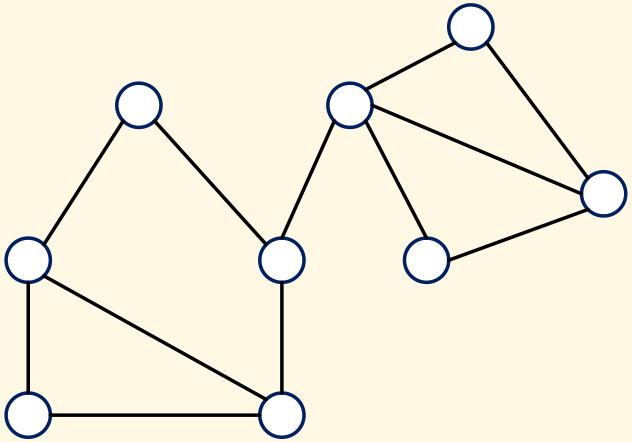
Paths

- A *path* p is an ordered list $\langle v_1, v_2, \ldots, v_k \rangle$ of vertices, where v_{i+1} is neighbor (in undirected graph, or out-neighbor in directed graph) of v_i for any $i = 2..k$
  - p is called a path from v_1 to v_k
- *Length* of a path is the number of edges on it
- Example: $\langle 1,2,7,9,5 \rangle$
  - Length: 4

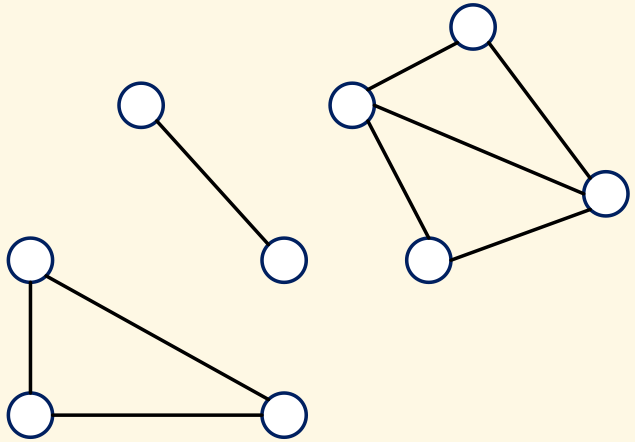


Connectivity

- Two vertices u, v are *connected* iff there exists at least one path from u to v; otherwise, they are disconnected
- A graph is called *connected* if every pair of vertices are connected; otherwise, it is *disconnected*
- Trees are connected graphs



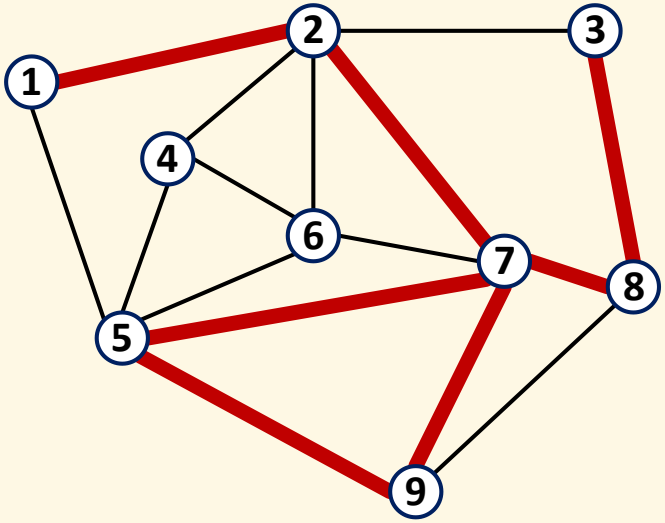
Connected graph

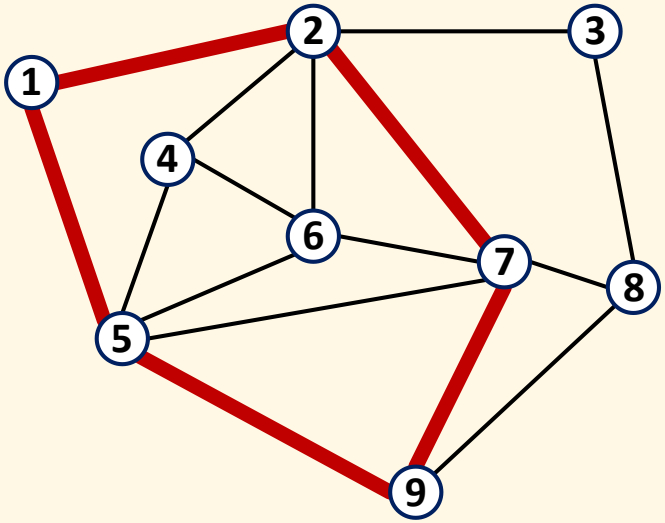


Disconnected graph

Simple Paths, Cycles

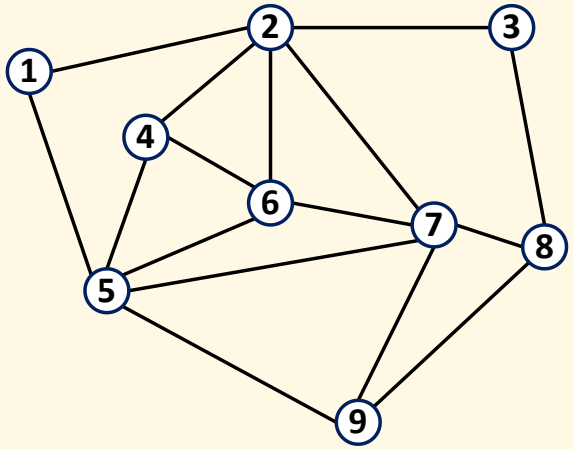
- A path is called *simple* iff all vertices, except possibly the first and the last, are distinct
  - Counter-example: ⟨1,2,7,9,5,7,8,3⟩
- A *cycle* is a simple path in which the first and the last vertices are identical
  - Example: ⟨1,2,7,9,5,1⟩



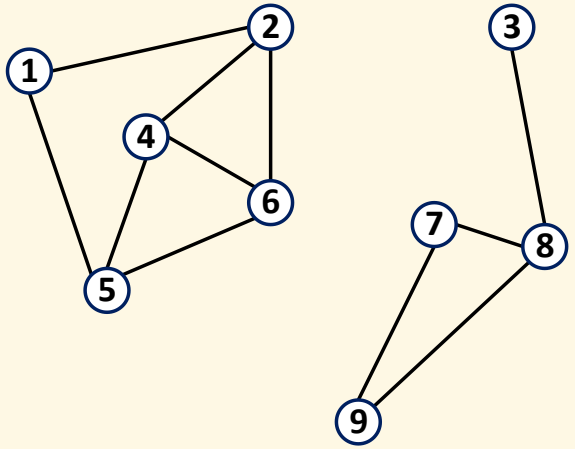


Cyclicity

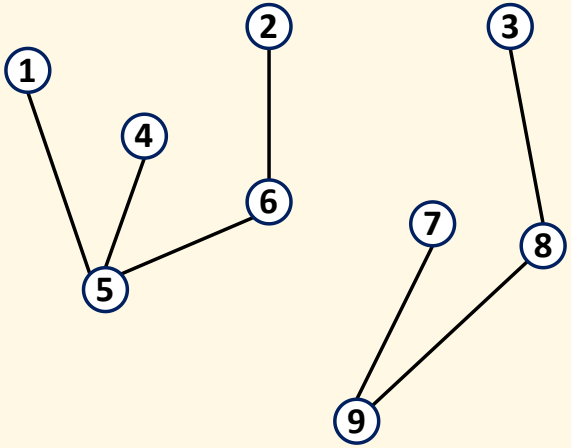
- A graph with cycles is called *cyclic*, otherwise, it is *acyclic*
  - A connected acyclic graph is called *tree*
  - A disconnected acyclic graph is called *forest*



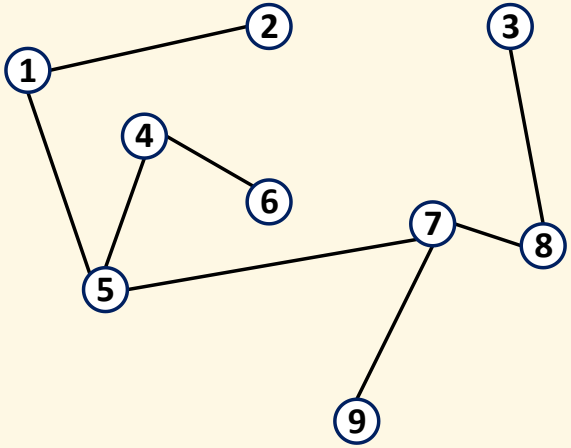
Connected cyclic graph



Disconnected cyclic graph



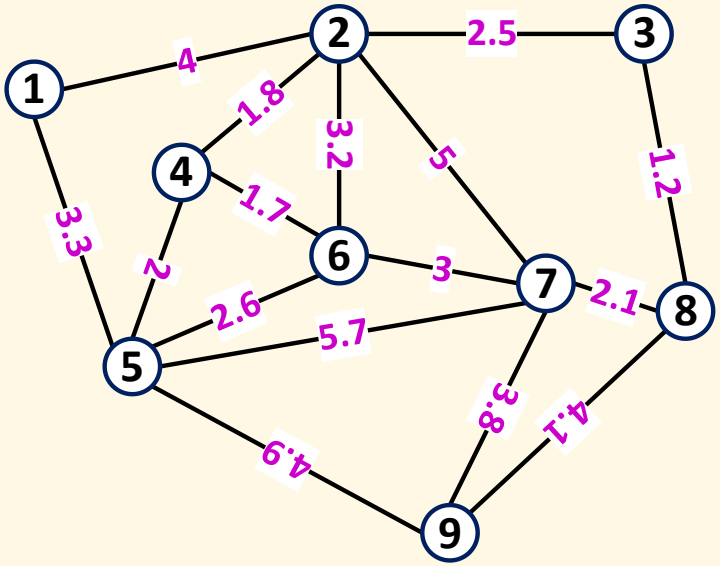
Disconnected acyclic graph (forest)



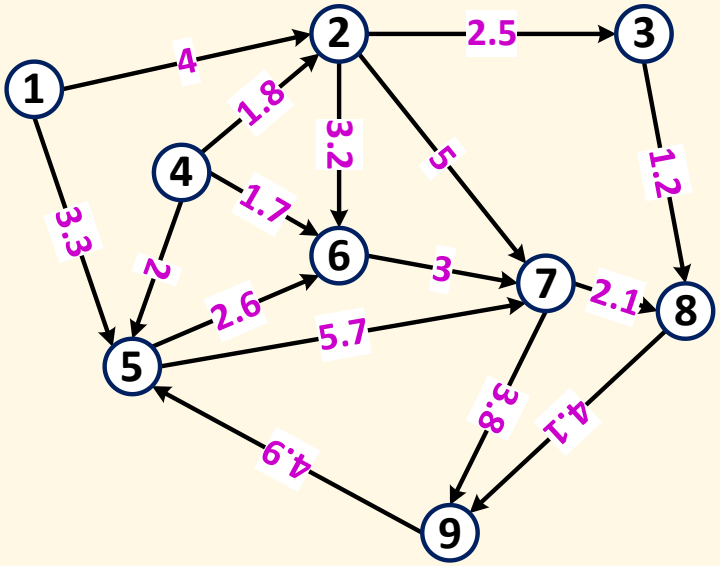
Connected acyclic graph (tree)

Weighted Graphs

- In a weighted graph, each edge is associated with a value called *weight* (or *cost*)



Undirected weighted graph

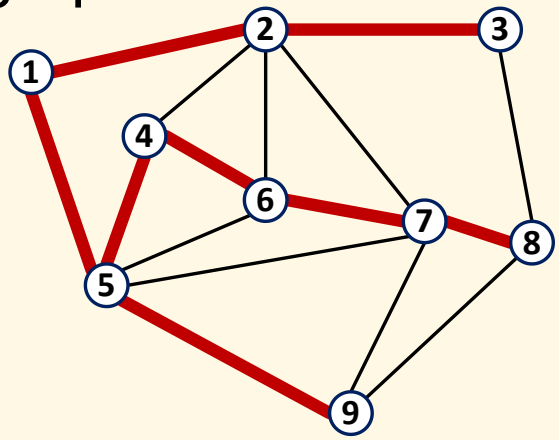


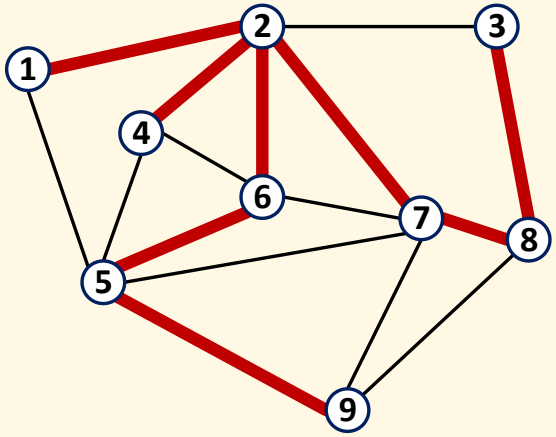
Directed weighted graph

- *Cost of a path* is summation of all the member edges

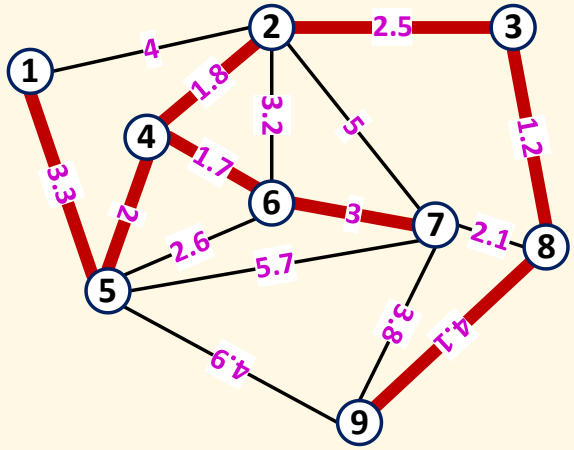
Spanning Tree

- Given a connected graph G, a *spanning tree* is an acyclic subgraph of G which covers all vertices of G





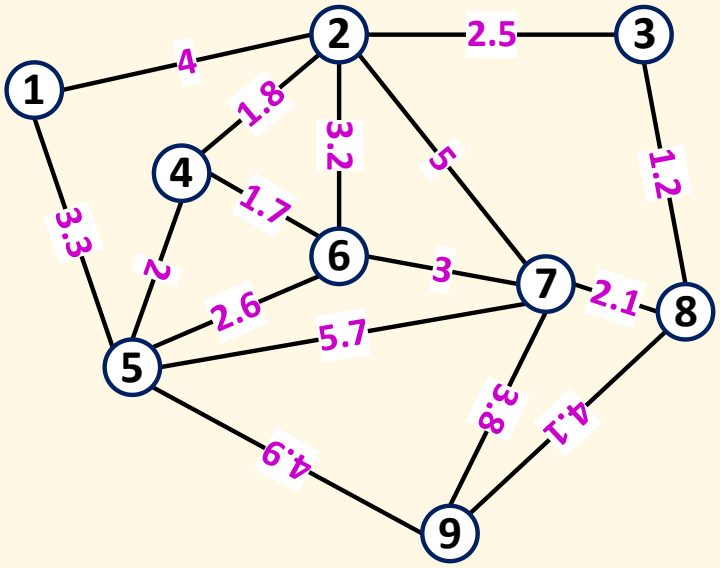
- There may exists multiple spanning trees for a given graph
- *Minimum spanning tree* is a spanning tree whose total edge weight is minimum



Graph Implementation using a Matrix
(Two-Dimensional Array, aka Adjacent Matrix)

Undirected Graph

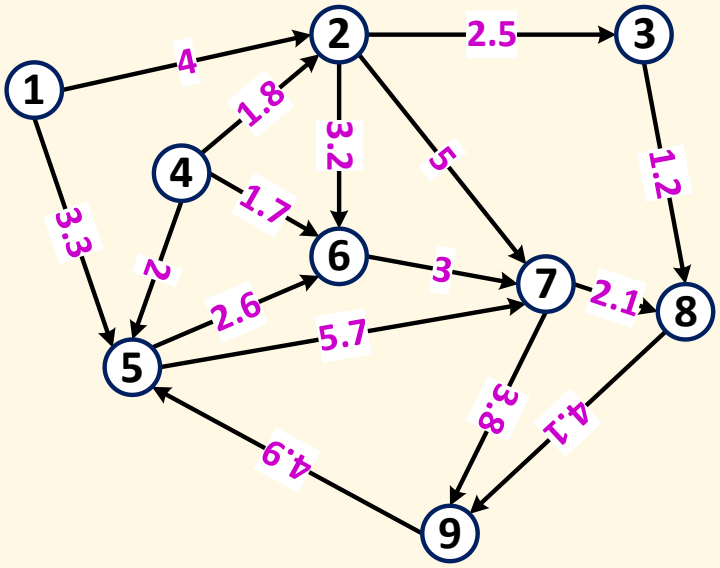
	1	2	3	4	5	6	7	8	9
1	0	4	∞	∞	3.3	∞	∞	∞	∞
2	4	0	2.5	1.8	∞	3.2	5	∞	∞
3	∞	2.5	0	∞	∞	∞	∞	1.2	∞
4	∞	1.8	∞	0	2	1.7	∞	∞	∞
5	3.3	∞	∞	2	0	2.6	5.7	∞	4.9
6	∞	3.2	∞	1.7	2.6	0	3	∞	∞
7	∞	5	∞	∞	5.7	3	0	2.1	3.8
8	∞	∞	1.2	∞	∞	∞	2.1	0	4.1
9	∞	∞	∞	∞	4.9	∞	3.8	4.1	0



- $a[i, j]$ represents the weight from vertex i to vertex j
  - Use a special value like -1, NULL or NaN to represent ∞
  - Symmetricity: $a[i, j] = a[j, i]$

Directed Graph

	1	2	3	4	5	6	7	8	9
1	0	4	∞	∞	3.3	∞	∞	∞	∞
2	∞	0	2.5	∞	∞	3.2	5	∞	∞
3	∞	∞	0	∞	∞	∞	∞	1.2	∞
4	∞	1.8	∞	0	2	1.7	∞	∞	∞
5	∞	∞	∞	∞	0	2.6	5.7	∞	∞
6	∞	∞	∞	∞	∞	0	3	∞	∞
7	∞	∞	∞	∞	∞	∞	0	2.1	3.8
8	∞	∞	∞	∞	∞	∞	∞	0	4.1
9	∞	∞	∞	∞	4.9	∞	∞	∞	0



- Symmetricity not satisfied

Implementation

- Be aware that array indices start from 0

```
double G[9][9] = {
   {0, 4, -1, -1, 3.3, -1, -1, -1, -1},
   {4, 0, 2.5, 1.8, -1, 3.2, 5, -1, -1},
   {-1, 2.5, 0, -1, -1, -1, -1, 1.2, -1},
   //...
};
```

- To evaluate an edge from vertex i to j: `G[i][j]`

Exercise

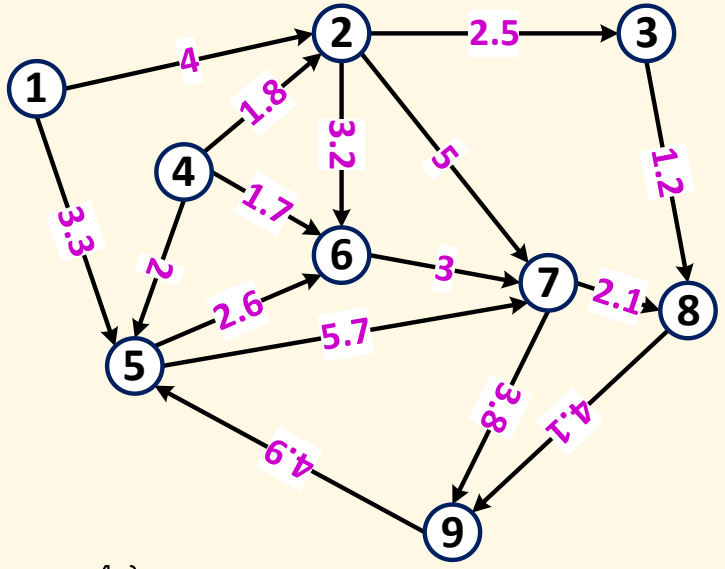
- Given a list of vertices: $P[0..k]$
  - Write the code to check if it is a path
  - Write the code to check if it is a simple path
  - Write the code to check if it is a cycle
  - Write the code to calculate the total cost of P
- Given two vertices i, j
  - Write the code to check if they are [in-, out-] neighbors
  - Write the code to check if they are connected
- Given a graph G
  - Write the code to check if it is cyclic
  - Write the code to check if it is connected

Graph Implementation using Separate Node and Edge Arrays/Lists

Example with Directed Graph

```
Vertex V[] = {
   {.name = "1"},
   {.name = "2"},
   //...
};

Edge E[] = {
   {.from = 0, .to = 1, .weight = 4},
   {.from = 0, .to = 4, .weight = 3.3},
   //...
};
```



Exercise

- Same as for matrix-based method

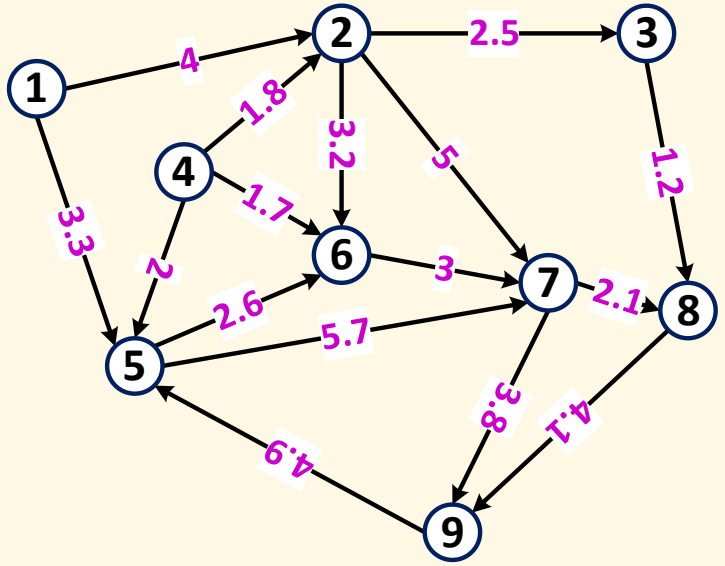
Graph Implementation using a Node Array with Embedded Edges

Example with Directed Graph

```
Vertex V[] = {
   {.name = "1", .edges = {
      {.to = 1, .weight = 4},
      {.to = 4, .weight = 3.3}
   }},

   {.name = "2", .edges = {
      {.to = 2, .weight = 2.5},
      {.to = 5, .weight = 3.2},
      {.to = 6, .weight = 5}
   }},

   //...
};
```



Exercise

- Same as for matrix-based method

Graph Search

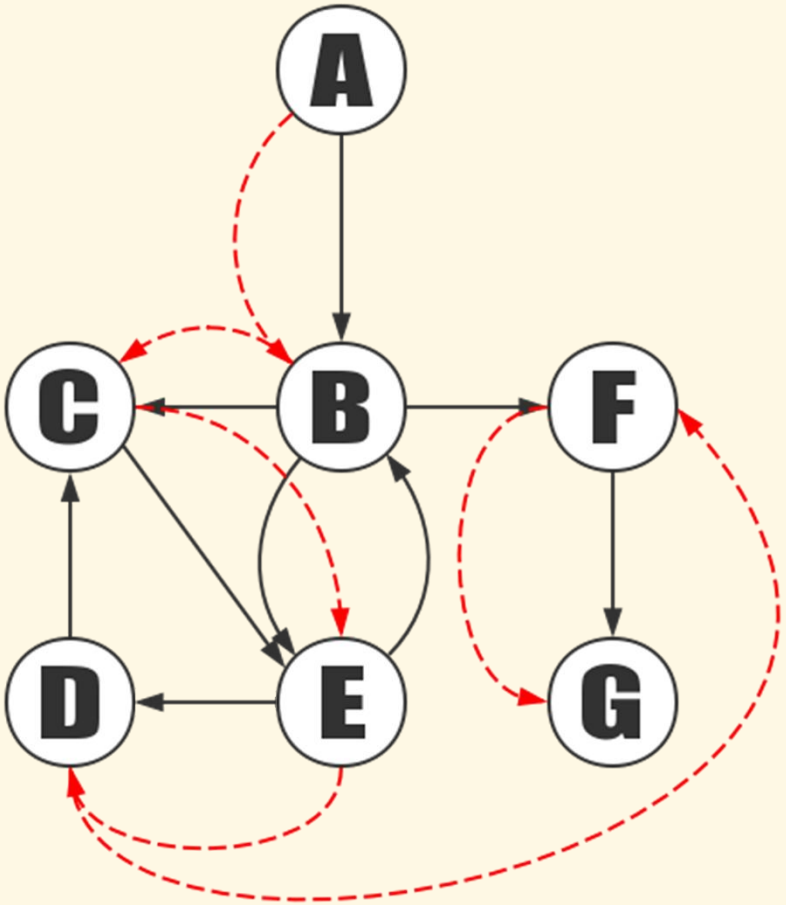
- Depth-first search (DFS)
- Breath-first search (BFS)
- Lowest-cost-first search (LCFS)

Graph Search Problem

- Given a graph G and a vertex v, find all vertices that can be reached from v, and (optionally) the best paths to those vertices

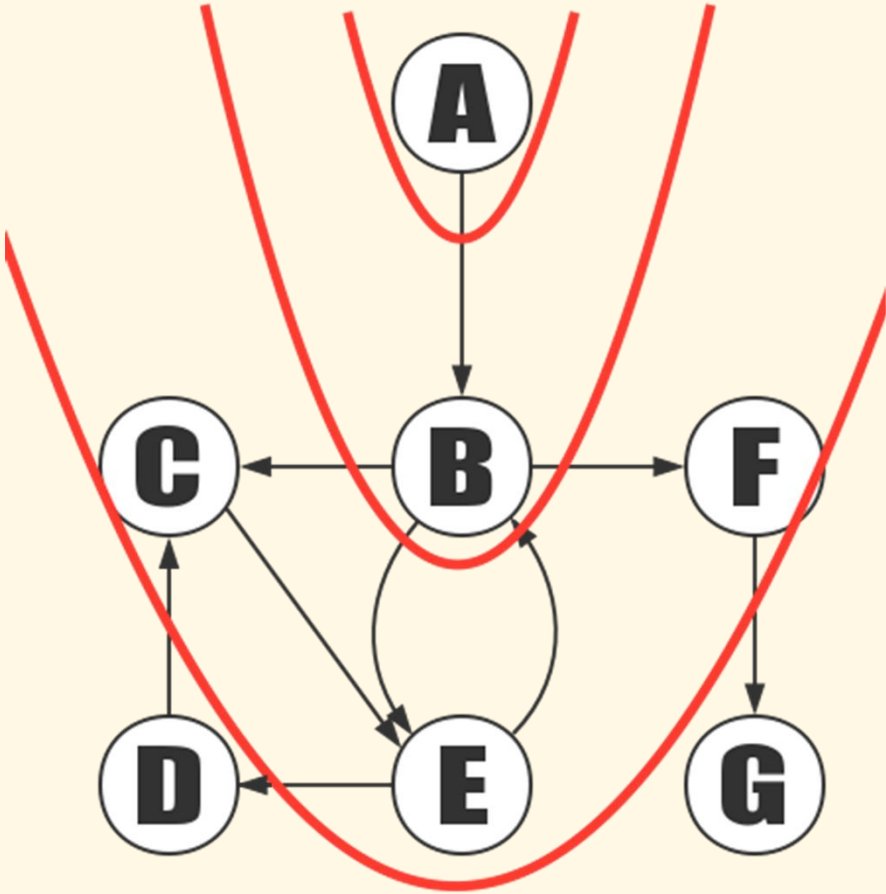
Depth-First Search (DFS)

- Algorithm:
  - Start with a vertex
  - Go to any vertex directly reachable from it that hasn't been visited
    - Mark the new vertex as visited
    - Explore the new vertex in a similar way
- 2 implementation methods:
  - Recursive
  - Iterative with help of a stack



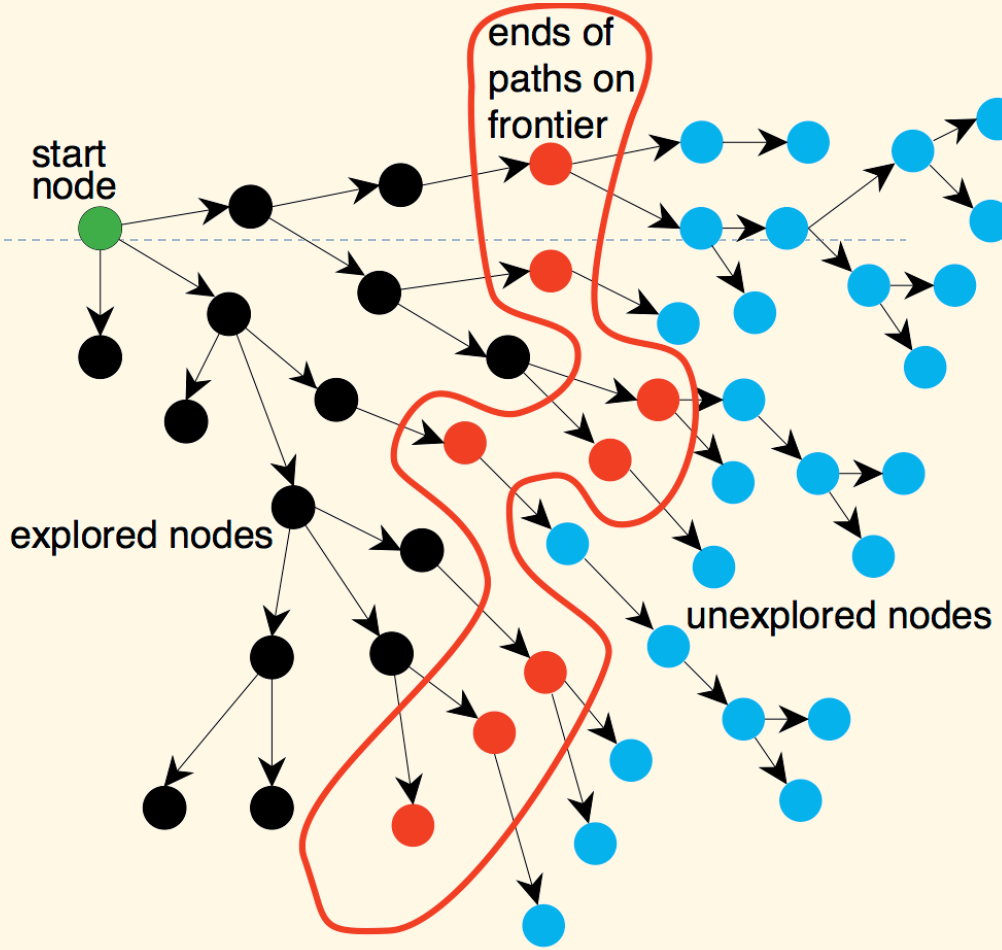
Breath-First Search (BFS)

- Algorithm:
  - Start with a vertex
  - Explore all directly reachable vertices from it that haven't been visited
    - Mark them as visited
    - Do similar process to these newly visited vertices
- Implementation method:
  - Iterative with help of a queue



Generic Search Algo

- 3 sets:
  - Explored nodes (black set)
  - Queued nodes (frontier, gray set)
  - Unexplored nodes (white set)
- Algorithm:



$\text{Frontier} := \{\langle s \rangle\}$
while $\text{Frontier} \neq \{\}$ **do**
  select and **remove** $\langle n_0, \ldots, n_k \rangle$ from Frontier
  if $\text{goal}(n_k)$ **then**
    return $\langle n_0, \ldots, n_k \rangle$
  $\text{Frontier} := \text{Frontier} \cup \{\langle n_0, \ldots, n_k, n \rangle : \langle n_k, n \rangle \in A\}$
return $\perp$

BFS and DFS as Special Cases

- From generic algorithm:
  - BFS: select = take first node from Frontier
  - DFS: select = take last node from Frontier

Lowest-Cost-First Search (Dijkstra's Algo)

- Idea: expand the wavefront to every possible paths by a constant speed until the target is reached

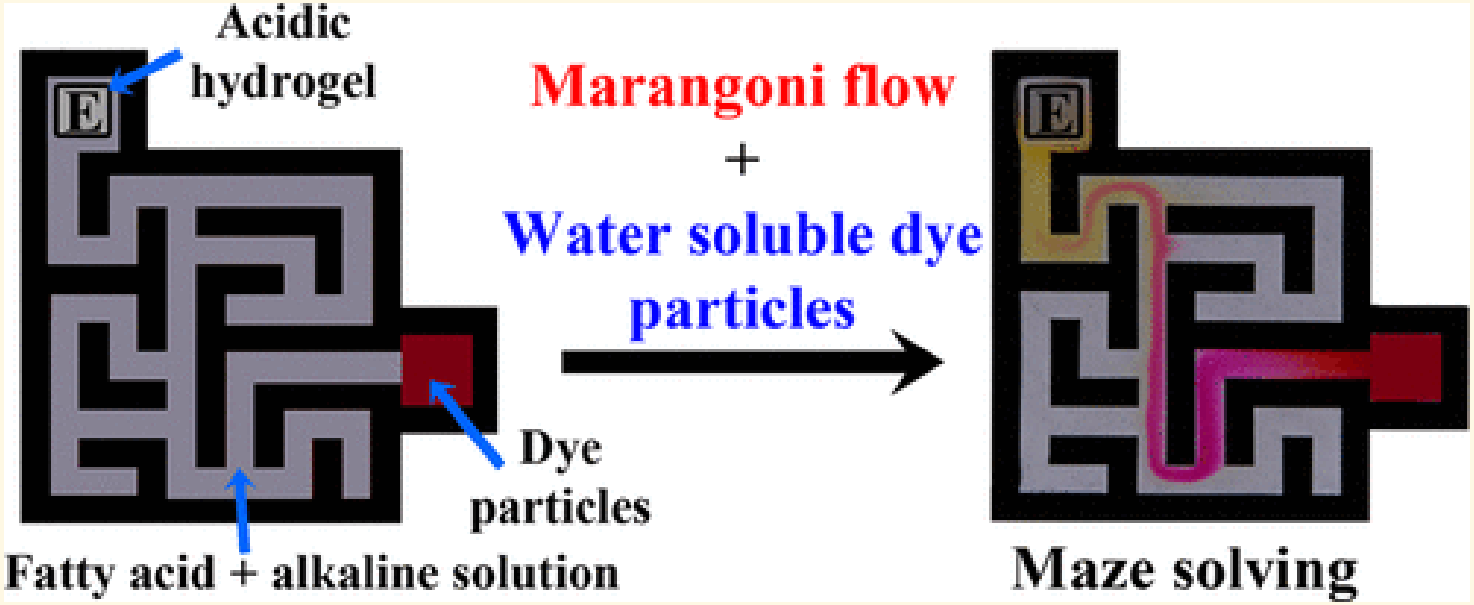
- From generic algorithm:
  - select = take node with lowest cost from Frontier
  - Frontier update: if a node is already included and the new cost is lower, replace its path
- Termination conditions:
  - When the target node is explored: to calculate path to one node
  - When Frontier is empty: to calculate path to every nodes

Dijkstra's Algo in Nature

- Lightning in extreme slow motion

Dijkstra's Algo in Nature

- Maze solving using fatty acid



Final Notes: Data Structures & Algorithms with C++ STL

Data Structures

- Dynamic array:
  - `std::vector<T>`
- Doubly linked list:
  - `std::list<T>`
- Stack and queue:
  - `std::stack<T>`
  - `std::queue<T>`
  - `std::deque<T>`
- Set:
  - `std::set<T>`
- Map:
  - `std::map<T>`
  - `std::multimap<T>`
- Tuple:
  - `std::pair<T>`
  - `std::tuple<T>`

Algorithms

- Sort:
  - `std::sort<RandomAccessIterator>(first, last)`
  - `std::sort<RandomAccessIterator, Compare>(first, last, comp)`
  - `std::list<T>::sort()`
  - `std::list<T>::sort<Compare>(comp)`
- Search:
  - `std::find<InputIterator, T>(first, last, val)`
  - `std::find_if<InputIterator, UnaryPredicate>(first, last, pred)`
  - `std::search<ForwardIterator>(first1, last1, first2, last2)`
  - `std::search<ForwardIterator, BinaryPredicate>(first1, last1, first2, last2, pred)`